

Corso di preparazione per la selezione territoriale delle Olimpiadi di Informatica

Lezione 6. Programmazione dinamica

Blerina Sinimeri



Esercizi dall'ultima volta

- Missioni segrete (regionali 2008) (dopo greedy, ricorsione: risolvo con programmazione dinamica!) <https://training.olinfo.it/#/task/missioni/statement>
- Sushi variegato (pre OII 2017) https://training.olinfo.it/#/task/preoii_yutaka/statement
- Esame di maturità (ABC 2016) https://training.olinfo.it/#/task/abc_esame/statement
- Numeri di Figonacci (OIS 2014) <https://training.olinfo.it/#/task/figonacci/statement>
- Minisuppli (LuissMatics 2018) https://training.olinfo.it/#/task/luiss_suppli/statement
- PROBLEMI IN SOSPESO:
- Sushi https://training.olinfo.it/#/task/preoii_sushi/statement
- Treni https://training.olinfo.it/#/task/preoii_treni/statement

Programmazione dinamica

How should I explain dynamic programming to a 4-year-old?

This question previously had details. They are now in a comment.

 **Jonathan Paulson** · January 3, 2013
Software Engineer at Jump Trading

Originally Answered: How should I explain what dynamic programming is to a 4-year-old?

writes down "1+1+1+1+1+1+1+1 =" on a sheet of paper

"What's that equal to?"

counting "Eight!"

writes down another "1+" on the left

"What about that?"

quickly "Nine!"

"How'd you know it was nine so fast?"

"You just added one more"

"So you didn't need to recount because you remembered there were eight! *Dynamic Programming* is just a fancy way to say 'remembering stuff to save time later'"

1.2M views · View Upvoters · View Sharers

 You, Feifei Wang and Jonathan Kotker upvoted this

 50.4K  77  166   



Programmazione dinamica



How should I explain dynamic programming to a 4-year-old?

This question previously had details. They are now in a comment.



Jonathan Paulson · January 3, 2013
Software Engineer at Jump Trading

Originally Answered: How should I explain what dynamic programming is to a 4-year-old?

writes down "1+1+1+1+1+1+1+1 =" on a sheet of paper

DP è un metodo in cui memorizziamo i valori calcolati in precedenza in modo da poterli recuperare facilmente di nuovo senza dover ricalcolare.

"What about that?"

quickly "Nine!"

"How'd you know it was nine so fast?"

"You just added one more"

"So you didn't need to recount because you remembered there were eight! *Dynamic Programming* is just a fancy way to say 'remembering stuff to save time later'"

1.2M views · View Upvoters · View Sharers



You, Feifei Wang and Jonathan Kotker upvoted this



50.4K



77



166



Il problema dello zaino (knapsack)

Molte varianti...
(versione non frazionaria)



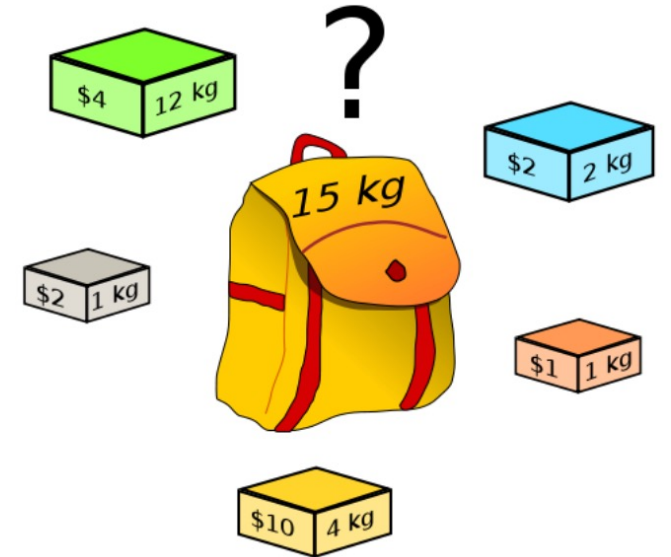
Il problema dello zaino (knapsack)

- Un ladro ha uno zaino che può contenere al più W Kg di peso
- Può rubare n oggetti (senza superare il limite di peso)
- Oggetto i pesa w_i Kg e ha un valore di v_i Euro
- Cosa gli conviene rubare? **Massimizzare valore complessivo della refurtiva senza superare il massimo peso W sopportabile dallo zaino**
- **Gli oggetti vanno presi per intero:** non è possibile inserire nello zaino *porzioni di oggetti!* (versione non frazionaria)



Il problema dello zaino (knapsack)

- Input: un zaino che può contenere al più W Kg di peso ed n oggetti, (v_i, w_i)
- Output: Scegliere un sottoinsieme S di oggetti tali che $\sum_{i \in S} v_i \leq W$ e tali che $\sum_{i \in S} w_i$ sia massima.



Un algoritmo greedy



- Idea: ad ogni passo scegli tra gli oggetti che possono ancora essere inseriti nello zaino, quello di peso massimo!
- Questo algoritmo funziona?

Un algoritmo greedy



- Idea: ad ogni passo scegli tra gli oggetti che possono ancora essere inseriti nello zaino, quello di peso massimo!
- Questo algoritmo funziona?

Supponiamo $W = 20kg$ e gli oggetti sono tre:

- $w_1 = 10kg, v_1 = 15 \text{ euro}$
- $w_2 = 8kg, v_2 = 20 \text{ euro}$
- $w_3 = 20kg, v_1 = 28 \text{ euro}$
- La soluzione del greedy : 28
- La soluzione ottima: 35!!

Un **altro** algoritmo greedy



- Idea: cerchiamo di migliorare l'algoritmo precedente: Ad ogni passo scegli tra gli oggetti che possono ancora essere inseriti nello zaino (cioè non oltrepassano la capacità dello zaino), quello che ha un valore maggiore e meno peso. Possiamo provare a misurare questo valore come $q_i = \frac{v_i}{w_i}$. Quindi ad ogni passo scelgo l'oggetto che può essere ancora inserito e ha q_i maggiore.
- Questo algoritmo funziona?

Un altro algoritmo greedy



- Idea: cerchiamo di migliorare l'algoritmo precedente: Ad ogni passo scegli tra gli oggetti che possono ancora essere inseriti nello zaino (cioè non oltrepassano la capacità dello zaino), quello che ha un valore maggiore e meno peso. Possiamo provare a misurare questo valore come $q_i = \frac{v_i}{w_i}$. Quindi ad ogni passo scelgo l'oggetto che può essere ancora inserito e ha q_i maggiore.
- Questo algoritmo funziona?

Supponiamo $W = 20kg$ e gli oggetti sono tre:

- $w_1 = 10kg, v_1 = 15 \text{ euro}$. $q_1 = 1.5$
- $w_2 = 8kg, v_2 = 20 \text{ euro}$ $q_2 = 2.5$
- $w_3 = 20kg, v_3 = 28 \text{ euro}$ $q_3 = 1.4$
- La soluzione del greedy prende q_2 e q_1 ! Funziona!
- Ma funziona sempre?

Un altro algoritmo greedy



- Idea: cerchiamo di migliorare l'algoritmo precedente: Ad ogni passo scegli tra gli oggetti che possono ancora essere inseriti nello zaino (cioè non oltrepassano la capacità dello zaino), quello che ha un valore maggiore e meno peso. Possiamo provare a misurare questo valore come $q_i = \frac{v_i}{w_i}$. Quindi ad ogni passo scelgo l'oggetto che può essere ancora inserito e ha q_i maggiore.
- Questo algoritmo funziona?

Supponiamo $W = 20kg$ e gli oggetti sono tre:

- $w_1 = 10kg, v_1 = 15 \text{ euro}, q_1 = 1.5$
- $w_2 = 10kg, v_2 = 20 \text{ euro}, q_2 = 2$
- $w_3 = 11kg, v_3 = 33 \text{ euro}, q_3 = 3$
- La soluzione del greedy prende q_3 !
- Sbaglia!

Un algoritmo di forza bruta ricorsivo?



- Algoritmo semplice richiede di enumerare tutti i possibili 2^n sottoinsiemi degli n elementi e scegliere la migliore combinazione (massimo beneficio rispettando il vincolo del peso)
- Se analizzate questo metodo noterete che ri-calcola troppe volte la stessa quantità.
- Algoritmo programmazione dinamica di solito molto meglio!
- Per PD: Idea: risolviamo utilizzando **sottoproblemi**.

Programmazione dinamica



Primo approccio:

- $OPT(i)$ = valore della soluzione ottima scegliendo un sottoinsieme degli oggetti $1, \dots, i$.
- Come posso passare da i ad $(i+1)$ oggetti? In altre parole conosco $OPT(i)$ posso conoscere $OPT(i+1)$?

Programmazione dinamica



Primo approccio:

- $OPT(i)$ = valore della soluzione ottima scegliendo un sottoinsieme degli oggetti $1, \dots, i$.
- Come posso passare da i ad $(i+1)$ oggetti? In altre parole conosco $OPT(i)$ posso conoscere $OPT(i+1)$?
- NON VA BENE perchè la soluzione ottima per gli oggetti $\{1, 2, \dots, i\}$ potrebbe non corrispondere alla soluzione ottima degli oggetti $\{1, 2, \dots, i, i+1\}$. In altre parole la soluzione per $OPT(i+1)$ potrebbe NON contenere la soluzione ottima del problema $OPT(i)$.

Programmazione dinamica



Esempio

Oggetto	v_i	w_i
1	10	3
2	4	8
3	9	9
4	11	8

Massimo peso: $W=20$

- La soluzione migliore per $\{1,2,3\}$ è : $\{1,2,3\}$ ($OPT(3)=23$)
- La soluzione migliore per $\{1,2,3,4\}$ è : $\{1,3,4\}$ ($OPT(4)=30$).
- In questo esempio la soluzione ottima utilizza la soluzione parziale $\{1, 3\}$, che è la soluzione ottima di $\{1, 2, 3\}$ quando il peso dello zaino è 12)

Programmazione dinamica



- Sottoproblemi definiti in termini di due parametri:
 - Insieme degli oggetti tra cui scegliere
 - Peso massimo ammissibile
- $OPT(i, p)$ = valore della soluzione ottima scegliendo un **sottoinsieme** degli oggetti **1, ..., i** ed assumendo che **p** è il massimo peso dello zaino.
- Riempiamo una tabella per memorizzare i valori per $OPT(i, p)$
- $i=1, 2, \dots, n$ e $p=0, 1, 2, \dots, W$

Programmazione dinamica

- $OPT(i, p)$ = valore della soluzione ottima scegliendo un **sottoinsieme** degli oggetti $1, \dots, i$ ed assumendo che p è il massimo peso dello zaino.
- $OPT(i, 0) = 0$ (se la capacità dello zaino è 0, allora non posso più inserire nulla !)
- $OPT(1, r) = ?$ Abbiamo un solo oggetto (v_1, w_1) :
 - Se $w_1 > r$ allora non posso inserire nessun oggetto e $OPT(1, r) = 0$
 - Se $w_1 \leq r$ allora posso inserire l'oggetto e $OPT(1, r) = v_1$
- $OPT(i, p) = ?$

Da i a $(i+1)$ oggetti...



Per calcolare $OPT(i,p)$

- 1) Se $w_{i+1} > p$, sicuramente l'oggetto $(i+1)$ non può essere preso, quindi $OPT(i+1,p) = OPT(i,p)$
- 2) Se $w_{i+1} \leq p$, possiamo inserire o meno l'oggetto:
 - Se lo inseriamo, $OPT(i+1,p) = v_{i+1} + OPT(i,p-w_{i+1})$
 - Se non lo inseriamo, $OPT(i+1,p) = OPT(i,p)$
 - Quale scelta è più conveniente? Prendiamo il **massimo** di queste due quantità!

Esempio



$$OPT(i, w) = \begin{cases} 0 & \text{if } i = 0 \\ OPT(i-1, w) & \text{if } w_i > w \\ \max \{ OPT(i-1, w), v_i + OPT(i-1, w - w_i) \} & \text{otherwise} \end{cases}$$

Esempio

i	v_i	w_i
1	1	1
2	6	2
3	18	5
4	22	6
5	28	7

$W = 11$

$$OPT(i, w) = \begin{cases} 0 & \text{if } i = 0 \\ OPT(i-1, w) & \text{if } w_i > w \\ \max\{OPT(i-1, w), v_i + OPT(i-1, w - w_i)\} & \text{otherwise} \end{cases}$$

Esempio

i	v_i	w_i
1	1	1
2	6	2
3	18	5
4	22	6
5	28	7

$W = 11$

Come si
trovano gli
oggetti
scelti?

$$OPT(i, w) = \begin{cases} 0 & \text{if } i = 0 \\ OPT(i-1, w) & \text{if } w_i > w \\ \max \{ OPT(i-1, w), v_i + OPT(i-1, w - w_i) \} & \text{otherwise} \end{cases}$$

	0	1	2	3	4	5	6	7	8	9	10	11
{ }	0	0	0	0	0	0	0	0	0	0	0	0
{ 1 }	0	1	1	1	1	1	1	1	1	1	1	1
{ 1, 2 }	0	1	6	7	7	7	7	7	7	7	7	7
{ 1, 2, 3 }	0	1	6	7	7	18	19	24	25	25	25	25
{ 1, 2, 3, 4 }	0	1	6	7	7	18	22	24	28	29	29	40
{ 1, 2, 3, 4, 5 }	0	1	6	7	7	18	22	28	29	34	35	40

Implementazione (bottom up iterativa)

KNAPSACK ($n, W, w_1, \dots, w_n, v_1, \dots, v_n$)

FOR $w = 0$ TO W

$M[0, w] \leftarrow 0.$

FOR $i = 1$ TO n

FOR $w = 0$ TO W

IF ($w_i > w$) $M[i, w] \leftarrow M[i-1, w].$

ELSE $M[i, w] \leftarrow \max \{ M[i-1, w], v_i + M[i-1, w - w_i] \}.$

RETURN $M[n, W].$

Implementazione (bottom up iterativa)

KNAPSACK ($n, W, w_1, \dots, w_n, v_1, \dots, v_n$)

FOR $w = 0$ TO W

$M[0, w] \leftarrow 0.$

$O(W)$

FOR $i = 1$ TO n

FOR $w = 0$ TO W

IF ($w_i > w$) $M[i, w] \leftarrow M[i-1, w].$

ELSE $M[i, w] \leftarrow \max \{ M[i-1, w], v_i + M[i-1, w - w_i] \}.$

$O(nW)$

RETURN $M[n, W].$

Implementazione (bottom up iterativa)

KNAPSACK ($n, W, w_1, \dots, w_n, v_1, \dots, v_n$)

FOR $w = 0$ TO W

$M[0, w] \leftarrow 0.$

$O(W)$

FOR $i = 1$ TO n

FOR $w = 0$ TO W

IF ($w_i > w$) $M[i, w] \leftarrow M[i-1, w].$

ELSE $M[i, w] \leftarrow \max \{ M[i-1, w], v_i + M[i-1, w - w_i] \}.$

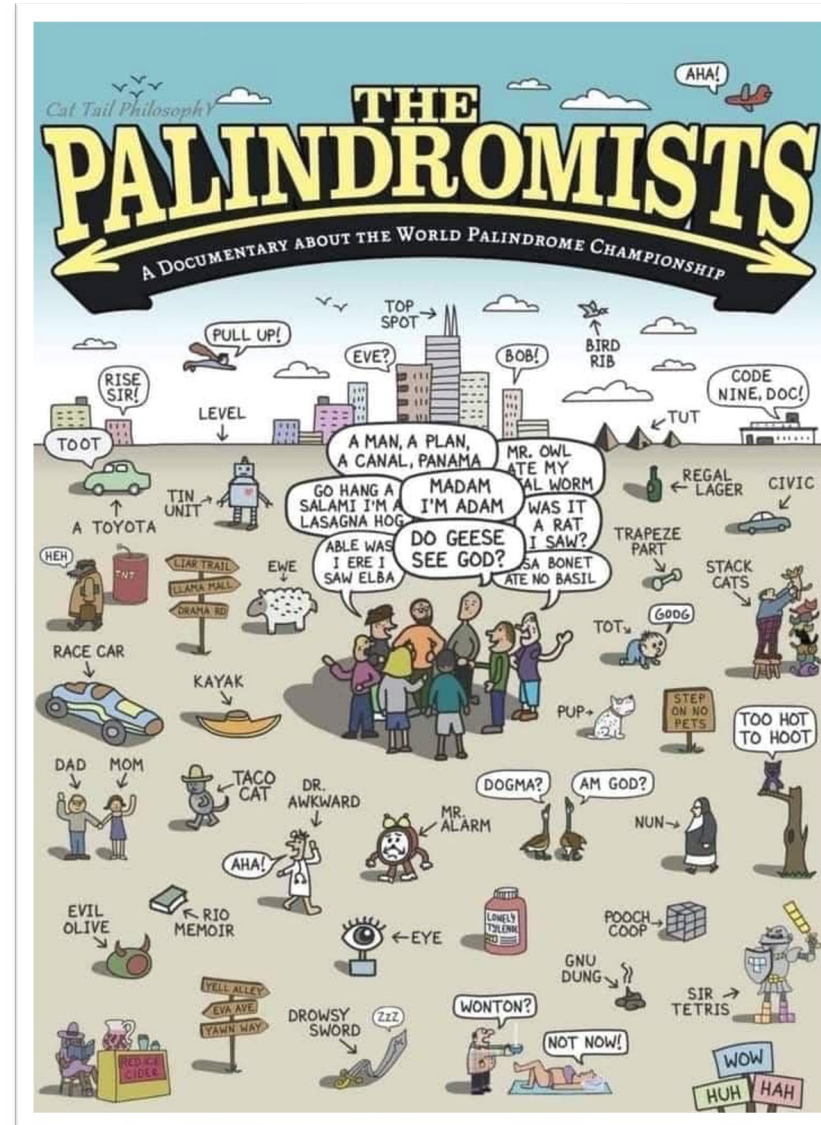
RETURN $M[n, W].$

Per dire se è polinomiale o meno dobbiamo guardare la dimensione dell'input:

- $w_1, w_2, \dots, w_n$
- $v_1, v_2, \dots, v_n$
- W
- Per rappresentare W abbiamo bisogno di $\log W$ bits.
- Quindi $nW = n \cdot 2^{\log W}$

$O(nW)$

Sottostringa palindroma



Esercizio: Sottostringa palindroma più lunga

- Una stringa si dice **palindroma** se rimane la stessa quando si legge da sinistra a destra e da destra a sinistra.
- Ad esempio **abcba**, **aba** sono entrambe stringhe palindrome, mentre **abcbca** non lo è.
- Scrive un programma che data una stringa S trova la più lunga sottostringa di S che è palindroma.
- Esempio: **abracadabra** la più lunga sottostringa palindroma ha lunghezza 3 (esempio : aca oppure ada)

Sottostringa palindroma: Programmazione dinamica

- $M[i,j] = \text{TRUE}$ se la sottostringa $S[i,j] = s_i \dots s_j$ è palindroma altrimenti FALSE.
- $i=0, \dots, n-1$ e $j=0, \dots, n-1$ dove $n = \text{lunghezza della stringa } S$
- $\max \{j - i + 1 : 0 \leq i \leq j \leq n - 1 \ \& \ M[i, j] = \text{True}\}$

Sottostringa palindroma: Programmazione dinamica

- $T[i,j] = \text{TRUE}$ se la sottostringa $S[i,j] = s_i \dots s_j$ è palindroma altrimenti FALSE.
- $i=0, \dots, n-1$ e $j=0, \dots, n-1$ dove $n = \text{lunghezza della stringa } S$
- $\max \{ j - i + 1 : 0 \leq i \leq j \leq n - 1 \ \& \ M[i,j] = \text{True} \}$
- $M[i,i] = \text{True}$
- $M[i,j] = ?$

Sottostringa palindroma: Programmazione dinamica

- $T[i,j] = \text{TRUE}$ se la sottostringa $S[i,j] = s_i \dots s_j$ è palindroma altrimenti FALSE.
 - $i=0, \dots, n-1$ e $j=0, \dots, n-1$ dove $n = \text{lunghezza della stringa } S$
 - $\max \{ j - i + 1 : 0 \leq i \leq j \leq n - 1 \ \& \ M[i,j] = \text{True} \}$
 - $M[i,i] = \text{True}$
 - $M[i,j] = ?$
- Una condizione necessaria perchè $M[i,j] = \text{TRUE}$ è che $x_i = x_j$
 - Se $j-i=1$ allora questa condizione è anche sufficiente.
 - Se invece $j-i > 1$ allora abbiamo un palindromo se $x_i = x_j$ E $M[i+1,j-1] = \text{TRUE}$

Sottostringa palindroma: Programmazione dinamica

- $T[i,j] = \text{TRUE}$ se la sottostringa $S[i,j] = s_i \dots s_j$ è palindroma altrimenti FALSE .
- $i=0, \dots, n-1$ e $j=0, \dots, n-1$ dove $n = \text{lunghezza della stringa } S$
- $\max \{j - i + 1 : 0 \leq i \leq j \leq n - 1 \ \& \ M[i,j] = \text{True}\}$
- $M[i,i] = \text{True}$

$$M[i, j] = \begin{cases} \text{TRUE} & \text{if } j - i \leq 1 \text{ and } x_i = x_j \\ M[i+1, j-1] & \text{if } j - i > 1 \text{ and } x_i = x_j \\ \text{FALSE} & \text{otherwise} \end{cases}$$

Algorithm *Palindrome(X)*

Require: X a n length sequence

Ensure: Returns the longest palindrome substring of X

```
1:  $M \leftarrow$  matrix of size  $n \times n$ 
2: for  $i \leftarrow 1$  to  $n$  do
3:    $M[i, i] \leftarrow \text{true}$ 
4: end for
5:  $max \leftarrow 1$ 
6: for  $j \leftarrow 0$  to  $n$  do
7:    $M[0, j] \leftarrow j$ 
8: end for
9: for  $k \leftarrow 1$  to  $n$  do
10:  for  $i \leftarrow 1$  to  $n - k$  do
11:     $j \leftarrow 1 + k$ 
12:    if  $(x_i == x_j)$  then
13:      if  $k == 1$  then
14:         $M[i, j] \leftarrow \text{true}$ 
15:      else
16:         $M[i, j] \leftarrow M[i - 1, j + 1]$ 
17:      end if
18:    else
19:       $M[i, j] \leftarrow \text{false}$ 
20:    end if
21:    if  $M[i, j] == \text{true} \wedge k > max$  then
22:       $max \leftarrow k + 1$ 
23:    end if
24:  end for
25: end for
26: return  $max$ 
```

Un problema di ACM-ICPC

Hello World! (helloworld)

<https://training.olinfo.it/-/task/helloworld/statement>



Hello World! (`helloworld`)

Limite di tempo: 1.0 secondi

Limite di memoria: 256 MiB

Franco vuole imparare a programmare, e per questo ha scritto il suo primo programma che stampa `helloworld`. Dopo averlo scritto, in preda all'entusiasmo, ha mandato in esecuzione molte volte il suo programma (mandando ogni volta il risultato su un file); sfortunatamente però, suo padre passa lì vicino e inciampa con il filo dell'alimentazione, spegnendo il PC.

Dopo aver riaccessato il PC, Franco nota che il file di testo si è danneggiato e adesso contiene una lunga stringa di caratteri mischiati. Invece di farsi prendere dal panico, Franco pensa ad un nuovo problema da risolvere: calcolare il numero di sottostringhe che cominciano per `hello` e finiscono per `world`.

Franco, essendo ancora alle prime armi, non riesce a risolvere il problema. Aiutalo nel suo intento!

Formato di input/output

Il tuo programma dovrà leggere da tastiera i seguenti dati:

- Riga 1: una singola stringa senza spazi composta da lettere dell'alfabeto minuscole.

Il tuo programma dovrà stampare a video i seguenti dati:

- un singolo numero: il numero di sottostringhe che iniziano per **hello** e terminano con **world**.

Assunzioni

- La stringa di input è lunga al massimo 1 000 000 di caratteri.

Esempi di input/output

stdin	stdout
helloworldishelloworld	3
jhelloworldandworld	2
worldhello	0

Soluzione del helloworld

- IDEA: Creo un vettore $T[i]$ =il numero delle occorrenze della parola “hello” nella stringa dalla posizione 0 alla posizione i .
- Inizializzo a 0 un contatore che conterrà il risultato finale.
- Scorro di nuovo la stringa a partire dalla posizione 0, se trovo una occorrenza della parola “world” che comincia nella posizione i , incremento il contatore di $T[i]$ in quanto questa sottostringa “world” andrà bene con tutte le “hello” fino a quella posizione.

```

#include <iostream>
#include <cmath>
using namespace std;

int main(){
    freopen("input.txt","r",stdin);
    freopen("output.txt","w",stdout);

    string w;
    cin >> w; //leggo la parola
    long long int count=0; // il numero delle sottostringhe che cominciano
    con hello e finiscono con world.
    int T[1000000]; // il numero delle parole hello che appaiono nella
    stringa w dalla posizione 0 alla pos i.
    int len=w.size();
    for (int i = 0; i < len ; i++){
        if (i<4) T[i]=0;
        else if (w.substr(i-4,5)=="hello")
            T[i]=1+T[i-1];
        else T[i]=T[i-1];
    }

    for (int i = 4; i < len ; i++){
        if (w.substr(i-4,5)=="world")
            count=count + T[i];
    }

    cout << count;
}

```

Problema del resto



Il problema del resto



- **Input:** Abbiamo dei tagli di monete: $v_1, v_2, \dots, v_k$. E un resto R da dare.
- **Output:** – Il più piccolo numero intero di monete necessarie per erogare un resto di n centesimi, supponendo che i tagli delle monete siano 50c, 20c, 10c, 5c, 2c e 1c – Supponiamo di avere una quantità finita di monete da poter erogare

Problema del resto: esempio



- Supponiamo di avere i tagli: 1,5,10 e 25 e voler dare il resto 63.
- $63 = 6 \times 10 + 3 \times 1 \rightarrow 9$ monete
- $63 = 2 \times 25 + 1 \times 10 + 3 \times 1 \rightarrow 6$ monete
- $63 = 63 \times 1 \rightarrow 63$ monete
-

Problema del resto: esempio



- Supponiamo di avere i tagli: 1, 5, 10 e 25 e voler dare il resto 63.
 - $63 = 6 \times 10 + 3 \times 1 \rightarrow 9$ monete
 - $63 = 2 \times 25 + 1 \times 10 + 3 \times 1 \rightarrow 6$ monete
 - $63 = 63 \times 1 \rightarrow 63$ monete
 -
- Attenzione: Se non c'è un taglio da 1 allora non è sempre possibile dare il resto.
 - Esempio: Dare 17 avendo a disposizione i tagli: 5, 10 e 25.
 - Assumiamo di avere sempre un taglio da 1 e di avere a disposizione una quantità illimitata di ciascun taglio.

Un algoritmo greedy

- Ordiniamo i tagli in modo decrescente:

$$v_1 > v_2 > \dots > v_k = 1$$

- Fino a quando $R > 0$ prendi il numero più grande possibile del taglio più grande da qui puoi prendere. Aggiorna R .
- Esempio: Tagli: $4 > 3 > 1$ e $R = 8$: $2 \times 4 \rightarrow \text{Greedy} = 2$ e $\text{Opt} = 2$

Un algoritmo greedy

- Ordiniamo i tagli in modo decrescente:

$$v_1 > v_2 > \dots > v_k = 1$$

- Fino a quando $R > 0$ prendi il numero più grande possibile del taglio più grande da qui puoi prendere. Aggiorna R .
- Esempio: Tagli: $4 > 3 > 1$ e $R=6$: $1 \times 4 + 2 \times 3 \rightarrow$ Greedy=3 ma Opt=2

Usiamo la programmazione dinamica

- Idea: Ad ogni passo decido se usare il taglio oppure no.
- Assumiamo che $1 = v_1 < v_2 < \dots < v_k$ e che R è un intero.
- $M[i,r]$ = il minimo numero di monete usando i tagli $v_1, \dots, v_i$ che servono per dare il resto r .
- $M[i,0]=0$ (se non devo dare nessun resto non devo dare nessuna moneta)
- $M[1,r]=r$ (se ho solo il taglio da 1 allora devo usarlo r volte).

Usiamo la programmazione dinamica

- $M[i,r]$ = il minimo numero di monete usando i tagli $v_1, \dots, v_i$ che servono per dare il resto r .
- $M[i,0]=0$ (se non devo dare nessun resto non devo dare nessuna moneta)
- $M[1,r]=r$ (se ho solo il taglio da 1 allora devo usarlo r volte).

$$M[k, R] = \begin{cases} M[k-1, R] & \text{if } V[k] > R \\ \min M[k-1, R], 1 + M[k, R - V[k]] & \text{otherwise} \end{cases}$$

Algorithm *DPMinimoMoneteResto*($R, V[], k$)

Require: Un resto R e un vettore V di dimensione k , contenente i tagli delle monete $1 = V[1] \leq V[2] \dots V[k]$.

Assumiamo che per ogni taglio abbiamo una disponibilità illimitata di quel taglio.

Ensure: Ritorna il numero minore di banconote per dare il resto R

```
1:  $M \leftarrow$  array di dimensione  $k \times (R + 1)$ 
2: for  $r \leftarrow 0$  to  $R$  do
3:    $M[1, r] \leftarrow r$ 
4: end for
5: for  $i \leftarrow 1$  to  $k$  do
6:    $M[i, 0] \leftarrow 0$ 
7: end for
8: for  $i \leftarrow 2$  to  $k$  do
9:   for  $r \leftarrow 1$  to  $R$  do
10:     $M[i, r] \leftarrow M[i - 1, r]$ 
11:    if ( $r \geq V[i]$ ) ( $1 + M[i, r - V[i]] < M[i, r]$ ) then
12:       $M[i, r] \leftarrow 1 + M[i, r - V[i]]$ 
13:    end if
14:  end for
15: end for
```

Provate a fare

Implementare un algoritmo che dati k tagli di banconote $1 = v_1 < v_2 < \dots < v_k$ e un valore R , calcola **il numero di modi diversi** di dare il resto R .

Ad esempio, se i tagli sono 1,4,5 e $R = 8$, ci sono 4 modi:

- 1,1,1,1,1,1,1
- 1,1,1,1,4
- 1,1,1,5
- 4,4

Un problema dalle OIS:

PILA DI MONETE

https://training.olinfo.it/-/task/tai_monete/statement



Pila di monete (monete)

Fabio e Giacomo, aprendo un vecchio forziere, hanno trovato N monete e decidono di giocare a **Pila di Monete**. Per giocare formano con le N monete un pila ed, uno alla volta, prendono dalla pila almeno 1 moneta ed al più K monete, fino a quando la pila non termina.

In **Pila di Monete** il punteggio di un giocatore si calcola sommando il valore delle monete che ha collezionato durante i suoi turni, aiuta Fabio, al quale spetta sempre il primo turno, a calcolare il punteggio massimo che può accumulare sapendo che Giacomo gioca in maniera ottimale.

Il file di input è composto da 2 righe, contenenti:

- Riga 1: gli interi N e K , separati da uno spazio.
- Riga 2: N interi, i valori delle monete.

Il file di output è composto da un'unica riga, contenente:

- Riga 1: il risultato della funzione `best_score` ossia il punteggio massimo che Fabio può ottenere.

Assunzioni

- $1 \leq N \leq 5000000$
- $K \leq N$
- $1 \leq monete_i \leq 180$

Esempi di input/output

input	output
5 3 3 7 2 8 1	11
5 5 1 2 3 4 5	15

Spiegazione

Nel **primo caso d'esempio** la strategia migliore per Fabio è prendere le prime 3 monete, sapendo che Giacomo prenderà le restanti.

Nel **secondo caso d'esempio** Fabio può prendere tutte le monete ottenendo un score pari a 15.

Esercizi per casa

- Rescaling sequence: https://training.olinfo.it/-/task/ois_rescaling/statement
- Studio amico: https://training.olinfo.it/-/task/abc_studioamico/statement
- Battaglia in pizzeria: https://training.olinfo.it/-/task/mat_pizzeria/statement
- Fibonacci colonies: https://training.olinfo.it/-/task/ois_fibonaccibug/statement
- Compulsive shopping: https://training.olinfo.it/-/task/ois_shopping/statement